24CSI024-DATA STRUCTURES AND ALGORITHMS

UNIT 2 LINEAR DATA STRUCTURES

STACK AND QUEUE

Pre Lab Task

Learn the following constructs:

1. Loops
2. Conditional Statements
3. Functions
4. File Handling

In Lab exercises

1. Implement

CODE:

```c
#include <stdio.h>
#include <ctype.h>
#define MAX 100
char charStack[MAX];
int charTop = -1;
int intStack[MAX];
int intTop = -1;
void pushChar(char x) {
    charStack[++charTop] = x;
}
char popChar() {
    return charStack[charTop--];
}
void pushInt(int x) {
    intStack[++intTop] = x;
}
int popInt() {
    return intStack[intTop--];
}
int precedence(char x) {
    if (x == '^')
        return 3;
    if (x == '*' || x == '/')
        return 2;
    if (x == '+' || x == '-')
        return 1;
    return 0;
}
void infixToPostfix() {
    char infix[MAX], postfix[MAX];
```

```c
    int i, j = 0;
    charTop = -1;
    printf("\nEnter an infix expression: ");
    scanf("%s", infix);
    for (i = 0; infix[i] != '\0'; i++) {
        if (isalnum(infix[i])) {
            postfix[j++] = infix[i];
        }
        else if (infix[i] == '(') {
            pushChar(infix[i]);
        }
        else if (infix[i] == ')') {
            while (charTop != -1 && charStack[charTop] != '(')
                postfix[j++] = popChar();
            if (charTop != -1)
                popChar();
        }
        else {
            while (charTop != -1 &&
                   charStack[charTop] != '(' &&
                   precedence(charStack[charTop]) >= precedence(infix[i])) {
                postfix[j++] = popChar();
            }
            pushChar(infix[i]);
        }
    }
    while (charTop != -1)
        postfix[j++] = popChar();
    postfix[j] = '\0';
```

```
    printf("Postfix expression: %s\n", postfix);
}
void evaluatePostfix() {
    char exp[MAX];
    int i, a, b, result;
    intTop = -1;
    printf("\nEnter a postfix expression: ");
    scanf("%s", exp);
    for (i = 0; exp[i] != '\0'; i++) {
        if (isdigit(exp[i])) {
            pushInt(exp[i] - '0');
        }
        else {
            if (intTop < 1) {
                printf("Invalid expression\n");
                return;
            }
            b = popInt();
            a = popInt();
            switch (exp[i]) {
                case '+':
                    result = a + b;
                    break;
                case '-':
                    result = a - b;
                    break;
                case '*':
                    result = a * b;
                    break;
```

```
                case '/':
                    if (b == 0) {
                        printf("Division by zero is not allowed\n");
                        return;
                    }
                    result = a / b;
                    break;
                case '^':
                    result = 1;
                    for (int j = 0; j < b; j++)
                        result *= a;
                    break;
                default:
                    printf("Invalid operator\n");
                    return;
            }
            pushInt(result);
        }
    }
    if (intTop != 0) {
        printf("Invalid expression\n");
        return;
    }
    printf("Result: %d\n", popInt());
}
```

```
void parenthesisMatching() {
    char exp[MAX];
    int i, valid = 1;
    charTop = -1;
    printf("\nEnter an expression: ");
    scanf("%s", exp);
    for (i = 0; exp[i] != '\0'; i++) {
        if (exp[i] == '(' || exp[i] == '[' || exp[i] == '{') {
            pushChar(exp[i]);
        }
        else if (exp[i] == ')' || exp[i] == ']' || exp[i] == '}') {
            if (charTop == -1) {
                valid = 0;
                break;
            }
            char x = popChar();
            if ((exp[i] == ')' && x != '(') ||
                (exp[i] == ']' && x != '[') ||
                (exp[i] == '}' && x != '{')) {
                valid = 0;
                break;
            }
        }
    }
    if (charTop != -1)
        valid = 0;
    if (valid)
        printf("Parentheses are balanced\n");
    else
        printf("Parentheses are not balanced\n");
```

```c
    else
}
int main() {
    int choice;
    while (1) {
        printf("\n===== STACK APPLICATIONS =====\n");
        printf("1. Infix to Postfix\n");
        printf("2. Postfix Expression Evaluation\n");
        printf("3. Parenthesis Matching\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                infixToPostfix();
                break;
            case 2:
                evaluatePostfix();
                break;
            case 3:
                parenthesisMatching();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}
```

OUTPUT:

(i) infix to postfix

```
===== STACK APPLICATIONS =====
1. Infix to Postfix
2. Postfix Expression Evaluation
3. Parenthesis Matching
4. Exit
Enter your choice: 1

Enter an infix expression: A+B*C
Postfix expression: ABC*+
```

(ii) expression evaluation

```
===== STACK APPLICATIONS =====
1. Infix to Postfix
2. Postfix Expression Evaluation
3. Parenthesis Matching
4. Exit
Enter your choice: 2

Enter a postfix expression: 23*54*+
Result: 26
```

(iii) parenthesis matching using Stack.

```
===== STACK APPLICATIONS =====
1. Infix to Postfix
2. Postfix Expression Evaluation
3. Parenthesis Matching
4. Exit
Enter your choice: 3

Enter an expression: {[()]}
Parentheses are not balanced
```

2. Implement stack and queue using Array and Linked List.

CODE:

```
            case 4:linkedQueue();break;
            case 5:return 0;
            default:printf("Invalid choice\n");
        }
    }
}
```

OUTPUT:

```
20 10
10 20
100 200
```

Post Lab exercises

1. Implementation of Circular Queue.